# Scalable Data Access

## DSC 102: Systems for Scalable Analytics

**Haojian Jin**

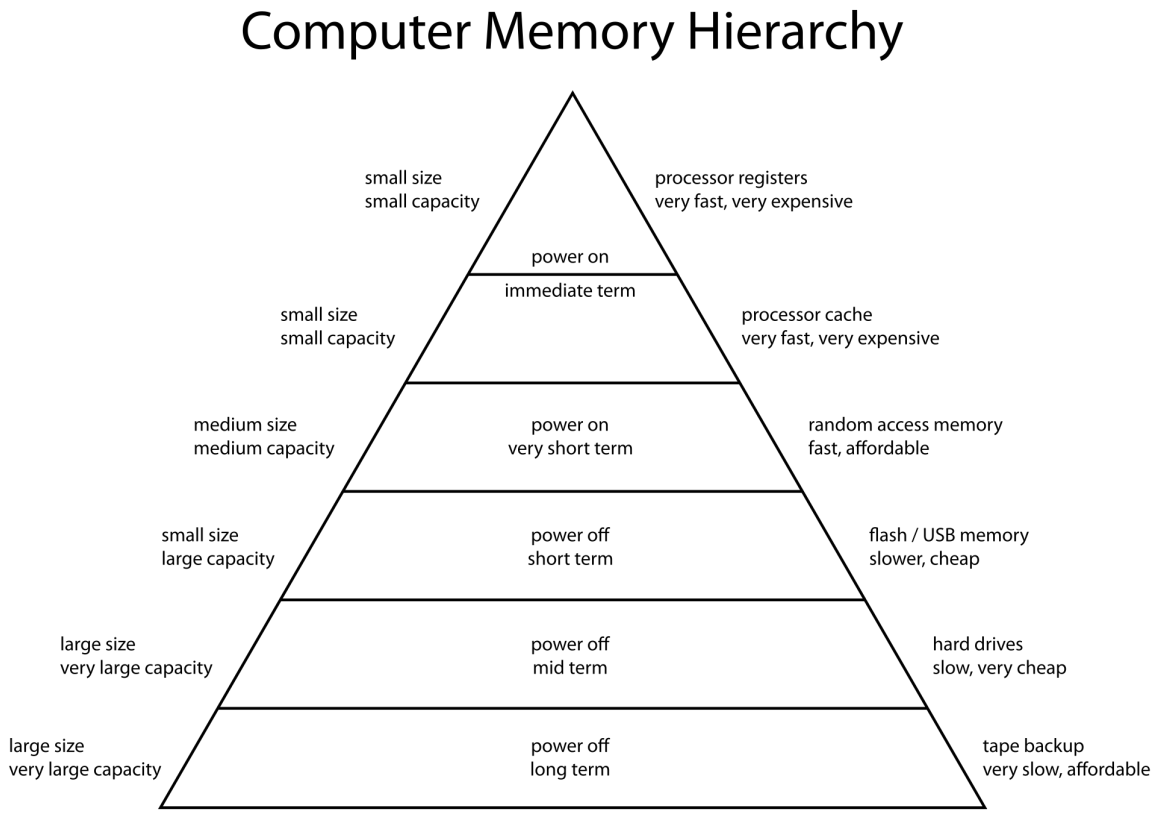
**UC San Diego**

# Where we are

- **L4:** the *work* was too big for one processor, so we split the **tasks** across workers
- **Today:** the *data* is too big for memory, so we split the **data** and stage it through the memory hierarchy

**Scalable data access: paged reads, I/O costs, layouts and access patterns, and how the core data science operations scale to disk.**

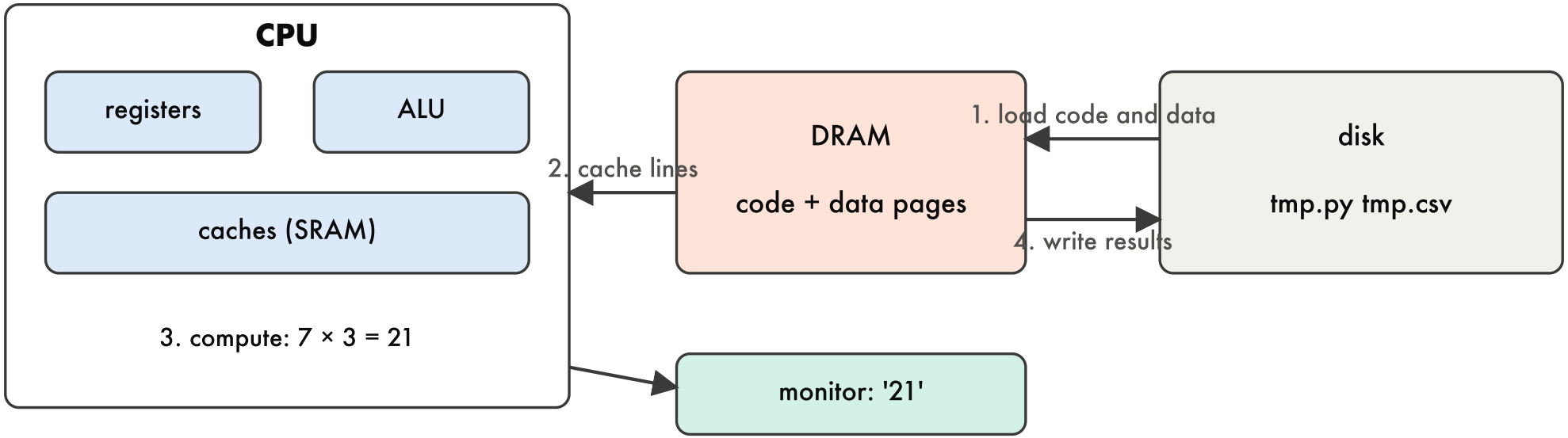
# Recap: the memory hierarchy




- Each level is **faster, smaller, and pricier** than the one below
- Caches: ~100 GB/s and MBs; DRAM: ~10 GB/s and tens of GB; SSD: ~GB/s and TBs; HDD: ~200 MB/s and tens of TB

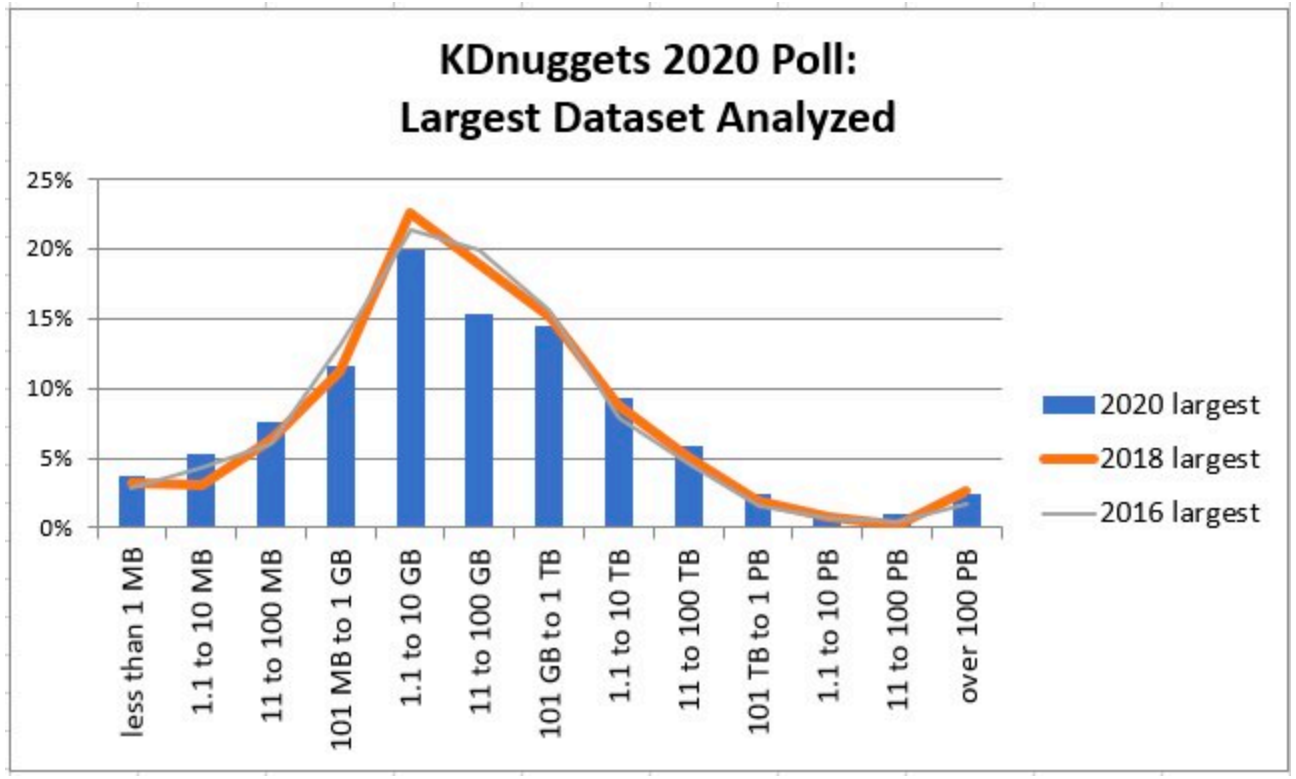
**"Does it fit in DRAM?" is the first question to ask about any dataset.**

# The memory hierarchy in action




**Q: what if the data in tmp.csv does not fit in DRAM?**

# Scale of datasets in practice




KDnuggets poll, 2020 (N = 562, 62% industry): *what is the largest dataset you have analyzed?*

- The peak sits between **1 GB and 100 GB**: right around a laptop's DRAM
- A long tail reaches past 100 TB

**Most of us live at the boundary where "fits in memory" stops being true.**

https://www.kdnuggets.com/2020/07/poll-largest-dataset-analyzed-results.html

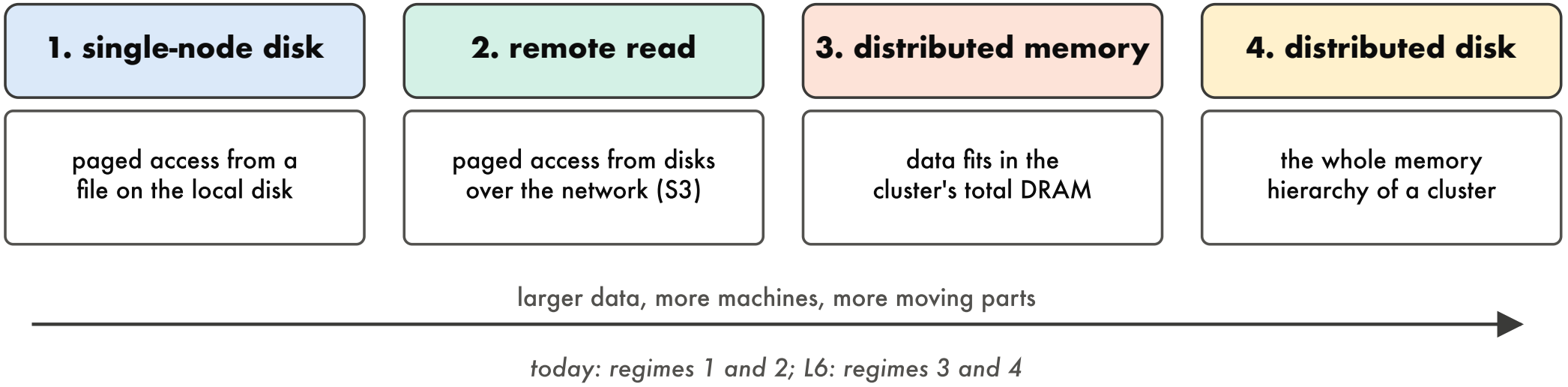
# Four questions for today

1. How does a program read a file **larger than DRAM**?
2. How do we **count the cost** of doing so?
3. How do the **data layout** and the **access pattern** interact?
4. How do the core **data science operations** scale to disk?

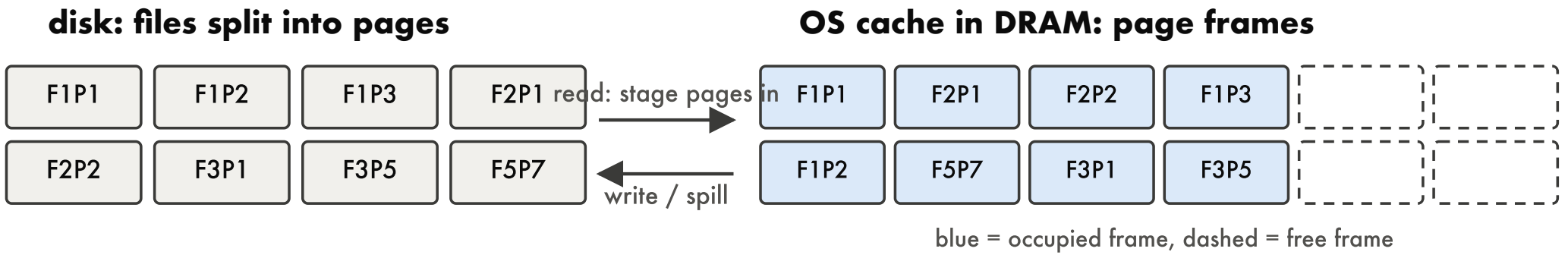
# Q1 · Scalable data access

**Central issue:** a large data file does not fit entirely in DRAM.

**Basic idea:** divide and conquer, again. "Split" the data file, virtually or physically, and **stage reads of its pages** from disk to DRAM (and vice versa for writes).



# Paged data access to DRAM




the file does not have to fit: the program sees pages come and go through the frames

DRAM is the cache; disk is the backing store; the OS (or the data system) manages both

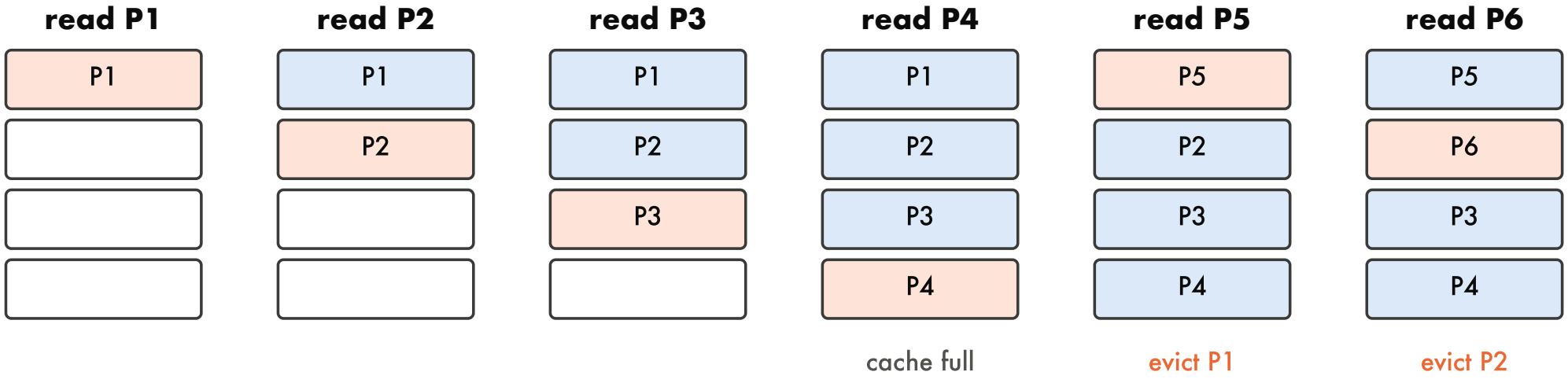
# Page management in the DRAM cache

| Term | Meaning |
|---|---|
| **Caching** | Retaining pages read from disk in DRAM |
| **Eviction** | Removing a page frame's content from DRAM |
| **Spilling** | Writing pages out from DRAM to disk |
| **Dirty page** | A cached page whose bytes were written; evicting it requires a spill |
| **Cache replacement policy** | The algorithm that picks which frame to evict when a new page must be cached and the cache is full |

- The set of DRAM-resident pages changes over the lifetime of a process
- Popular policies: **least recently used (LRU)**, most recently used, …

# Scaling to local disk: a filescan

**filescan of P1 … P6 through an OS cache with 4 frames (initially empty)**




each page is brought in exactly once: total I/O cost = 6 page reads

with a cache replacement policy (here LRU) deciding what to evict

# Scaling to local disk in practice

In general, scalable programs **stage access to pages** of a file on disk and use the available DRAM efficiently.

- Recall that DRAM size ≪ disk size
- Modern machines have tens of GB of DRAM, so read a **chunk** of the file at a time: thousands of pages, not one
- On HDDs, chunking means more **sequential I/O**: higher throughput, lower latency
- Similarly, write a chunk of dirtied pages at a time

**Page-at-a-time is the model; chunk-at-a-time is the implementation.**

# Scaling with remote reads

Same idea, but the disk is not local: stage page reads from remote disks over the **network** (for example from **S3**).

- More restrictive than local disk: **spilling** is impossible, or costs network I/O
- Fine for a **one-shot filescan** access pattern
- Use DRAM as the cache, with a replacement policy
- Can also use a smaller **local disk as a cache**: you did this in PA1

**The network is one more level of the hierarchy, slower than the local disk.**

# Q2 · Quantifying I/O: disk and network

Page reads and writes between DRAM and disk incur latency. We count them abstractly:

- **Disk I/O cost:** the number of page I/Os (maps to bytes given the page size)
- **Communication / network I/O cost:** the number of pages or bytes sent or received over the network

The cost is abstract; the mapping to **latency** is hardware-specific.

| **Example: a 40 GB file, 4 KB pages** | |
|---|---|
| I/O cost to read the file | **10 million** page I/Os |
| On a disk at 800 MB/s | 40 GB / 800 MB/s = **50 s** |
| Over a network at 200 MB/s | 40 GB / 200 MB/s = **200 s** |

# Q3 · Data layouts and access patterns

- **Data layout:** the order in which data is laid out on storage; a property of the **physical level**
- **Data access pattern:** the order in which a program needs the data; a property of the **program**
- Together they decide **what subset gets cached** in the higher levels of the hierarchy

**Key principle:** optimizing the data layout on disk for the data access pattern reduces I/O cost and latency.

Applies to HDDs and SSDs, but especially to HDDs because of their random-vs-sequential access gap.

# Row-store versus column-store layouts

| A | B | C | D |
|---|---|---|---|
| 1a | 1b | 1c | 1d |
| 2a | 2b | 2c | 2d |
| 3a | 3b | 3c | 3d |
| 4a | 4b | 4c | 4d |
| 5a | 5b | 5c | 5d |
| 6a | 6b | 6c | 6d |

**R**

*a page holds 4 cell values*

**Row-store (6 pages):**

| | | |
|---|---|---|
| 1a,1b,1c,1d | 2a,2b,2c,2d | 3a,3b,3c,3d |
| 4a,4b,4c,4d | 5a,5b,5c,5d | 6a,6b,6c,6d |

**Column-store (8 pages):**

| | | | |
|---|---|---|---|
| 1a,2a,3a,4a | 5a,6a | 1b,2b,3b,4b | 5b,6b |
| 1c,2c,3c,4c | 5c,6c | 1d,2d,3d,4d | 5d,6d |

A common dichotomy when serializing 2-D structured data (relations, matrices, DataFrames) to a file.

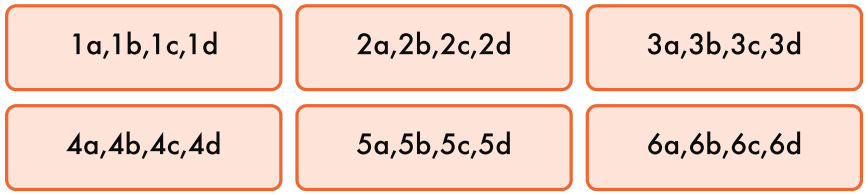
# The I/O cost of a sum over column B

| A | B | C | D |
|---|---|---|---|
| 1a | 1b | 1c | 1d |
| 2a | 2b | 2c | 2d |
| 3a | 3b | 3c | 3d |
| 4a | 4b | 4c | 4d |
| 5a | 5b | 5c | 5d |
| 6a | 6b | 6c | 6d |

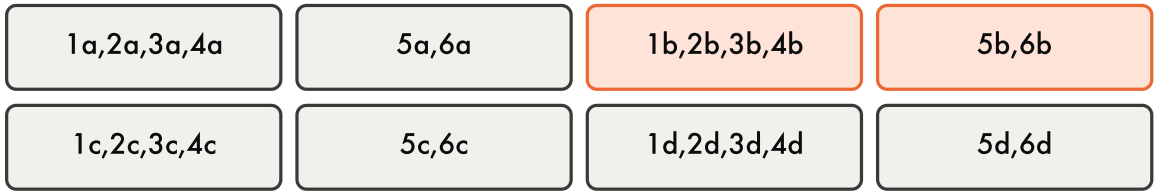
R

**SELECT SUM(B) FROM R**

**Row-store: all 6 pages read**




**Column-store: only B's 2 pages read**




**Depending on the access pattern, row-store and column-store I/O costs can be orders of magnitude apart.** The same holds for tensors.

# Hybrid, tiled, "blocked" layouts

| A | B | C | D |
|---|---|---|---|
| 1a | 1b | 1c | 1d |
| 2a | 2b | 2c | 2d |
| 3a | 3b | 3c | 3d |
| 4a | 4b | 4c | 4d |
| 5a | 5b | 5c | 5d |
| 6a | 6b | 6c | 6d |

**R**

*2×2 tiles, 4 cells per page*

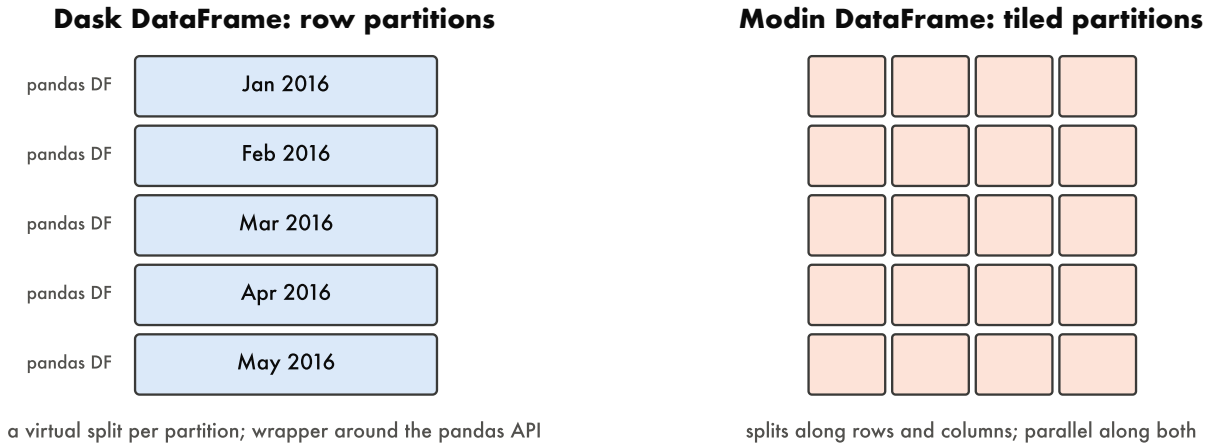
**Tiled store (6 pages):**

| | | |
|---|---|---|
| 1a,1b,2a,2b | 1c,1d,2c,2d | 3a,3b,4a,4b |
| 3c,3d,4c,4d | 5a,5b,6a,6b | 5c,5d,6c,6d |

a tile holds a 2-row × 2-column block:
reads along rows and along columns both touch half the pages

Common in analytical RDBMSs and in matrix and tensor systems. **Which layout yields lower I/O cost depends on the access pattern of the program.**

# Example: Dask's DataFrame




- **Dask DataFrame:** a **row-store** of virtual splits, each a pandas DataFrame, with the Dask API wrapping pandas per split; too large for DRAM means a manual `repartition()` to splits under ~1 GB
- **Modin DataFrame:** a **tiled store** that scales along both dimensions and parallelizes more easily, as SAP HANA, TimesTen, and ScaLAPACK did before

# Q4 · Scaling data science operations

Scalable data access for representative operations that are ubiquitous in data science:

**Database systems**

- Select
- Non-deduplicating project
- Simple SQL aggregates
- GROUP BY aggregates

**ML systems**

- Matrix sums and norms
- Batch gradient descent
- Stochastic gradient descent

**The question for each: what is the access pattern, and what is the I/O cost?**

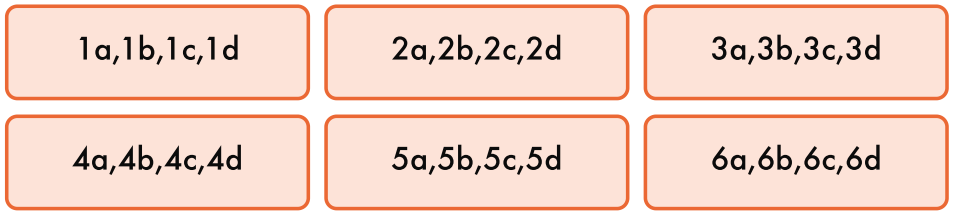
# Non-deduplicating project on a row-store

| A | B | C | D |
|---|---|---|---|
| 1a | 1b | 1c | 1d |
| 2a | 2b | 2c | 2d |
| 3a | 3b | 3c | 3d |
| 4a | 4b | 4c | 4d |
| 5a | 5b | 5c | 5d |
| 6a | 6b | 6c | 6d |

**R**

**SELECT C FROM R**

**Row-store: filescan, 6 reads**




read a page, keep only the C values, drop the rest on the fly;
I/O cost = 6 (read) + output pages (write)

- A straightforward **filescan**: read one page at a time into DRAM, with cache replacement if needed
- Drop the unneeded columns from the tuples on the fly

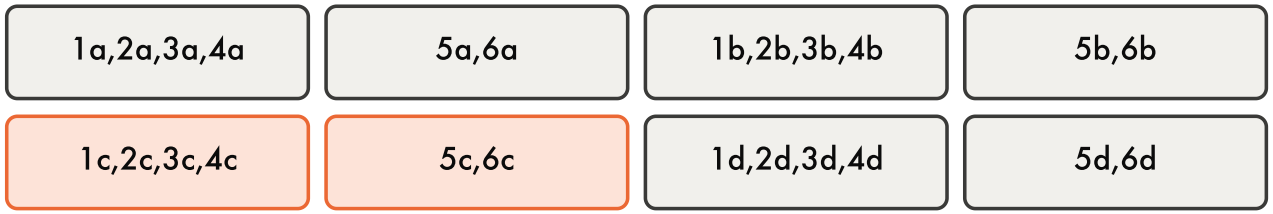
# Non-deduplicating project on a column-store

| A | B | C | D |
|---|---|---|---|
| 1a | 1b | 1c | 1d |
| 2a | 2b | 2c | 2d |
| 3a | 3b | 3c | 3d |
| 4a | 4b | 4c | 4d |
| 5a | 5b | 5c | 5d |
| 6a | 6b | 6c | 6d |

**R**

**SELECT C FROM R**

**Column-store: only C's pages, 2 reads**




I/O cost = 2 (read) + output pages (write)

- Since we only need column C, there is no need to read the other pages
- A big advantage of column-stores for SQL analytics queries (projects, aggregates): "OLAP"
- The rationale for column-store RDBMSs such as **Vertica**, and for **Parquet**

# Simple aggregates

```
SELECT MAX(A) FROM R
```

- Again a straightforward **filescan** access pattern, with one running value in DRAM
- The same I/O behavior as the non-deduplicating project:

| Layout | I/O cost |
|---|---|
| Row-store | **6** (read) + output pages (write) |
| Column-store | **2** (read) + output pages (write): only A's pages |

**One number in DRAM, one pass over the file.**

# Relational select

`SELECT C FROM R WHERE B = '3b'`, i.e. $\pi_C(\sigma_{B='3b'}(R))$

- A straightforward **filescan**: read pages or chunks from disk to DRAM one by one
- The CPU applies the **predicate** to the tuples in each page in DRAM
- Copy the satisfying tuples to temporary **output pages**
- Use LRU for cache replacement, if needed

**I/O cost: 6 (read) + output pages (write).**

# GROUP BY aggregate

| A | B | C | D |
|---|---|---|---|
| a1 | 1b | 1c | 4 |
| a2 | 2b | 2c | 3 |
| a1 | 3b | 3c | 5 |
| a3 | 4b | 4c | 1 |
| a2 | 5b | 5c | 10 |
| a1 | 6b | 6c | 8 |

**R**

**Row-store: one filescan**

| | | |
|---|---|---|
| a1,1b,1c,4 | a2,2b,2c,3 | a1,3b,3c,5 |
| a3,4b,4c,1 | a2,5b,5c,10 | a1,6b,6c,8 |

**SELECT A, SUM(D) FROM R GROUP BY A**

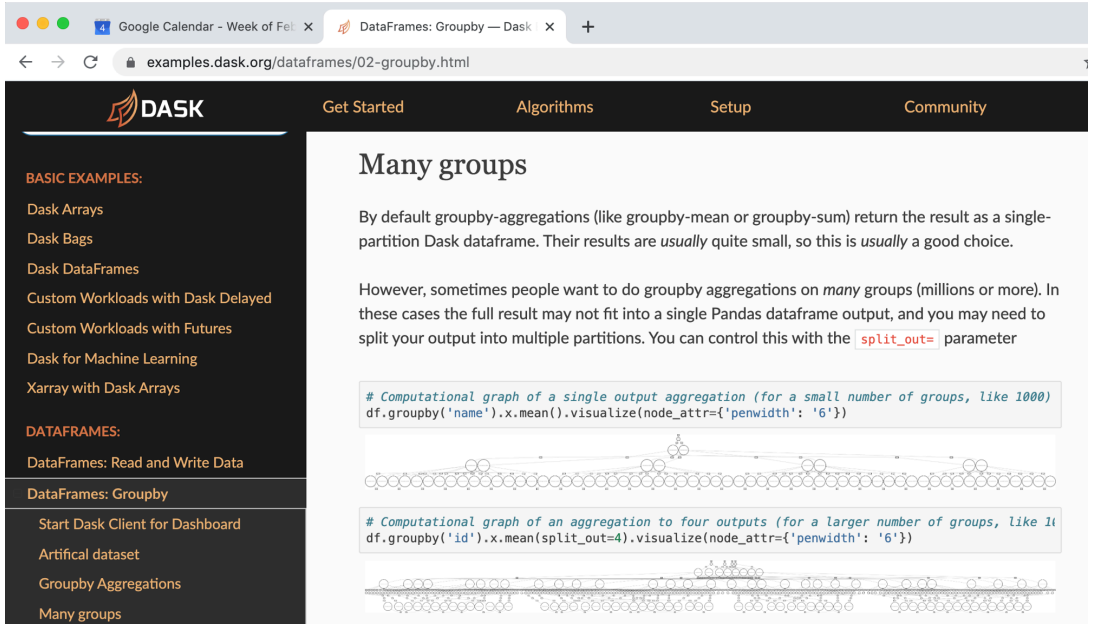
**hash table in DRAM, one record per group**

| A | running sum |
|---|---|
| a1 | 4 → 9 → 17 |
| a2 | 3 → 13 |
| a3 | 1 |

I/O cost = 6 + output

- We must **collect** each group's tuples and aggregate per group: a **hash table in DRAM**, one record per group, built during one filescan, *is* the output

# GROUP BY when the hash table exceeds DRAM




**Q: What if the hash table > DRAM?** The program likely **crashes**, or the OS **thrashes**, swapping its pages to and from disk.

**Q: How to scale to many groups? Divide and conquer:** split R on the values of A so each split's hash table fits; shrink the running information where possible. Dask's `split_out` does exactly this.

https://examples.dask.org/dataframes/02-groupby.html · CSE 132C covers how GROUP BY is scaled in RDBMSs.

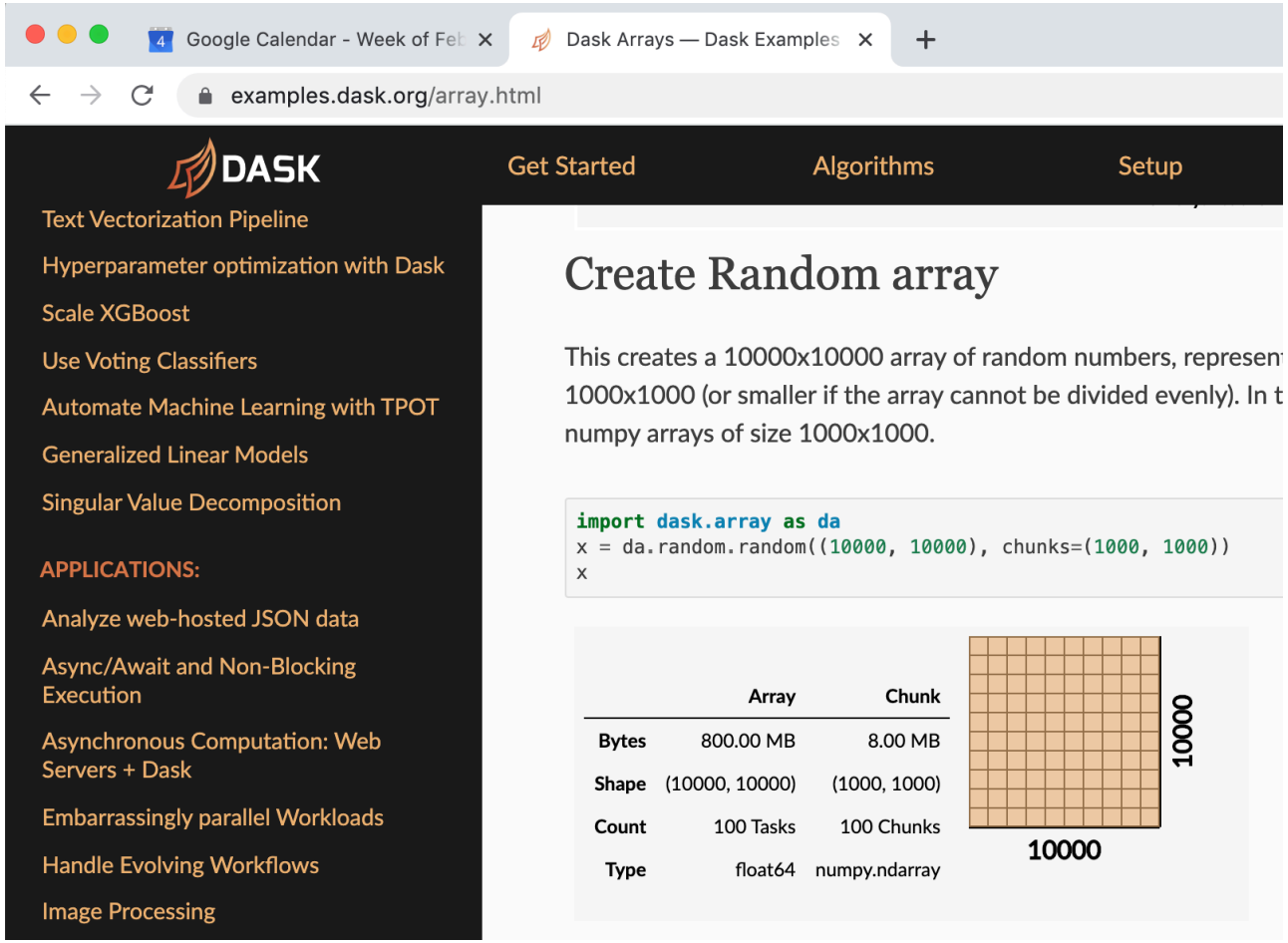
# Matrix sums and norms

$\|M\|_F^2 = \sum_{i,j} M_{ij}^2$ over a 6×4 matrix stored as six row pages of four cells.

- Again a straightforward **filescan**, very similar to a relational simple aggregate
- Running information in DRAM: the sum of squares so far: 0 → 5 → 10 → 15 → 20 → 30 → 40
- I/O cost: **6** (read) + output pages (write)

**Q: Column-store and tiled-store also cost 6 here. Why?**

# Scalable matrix and tensor algebra

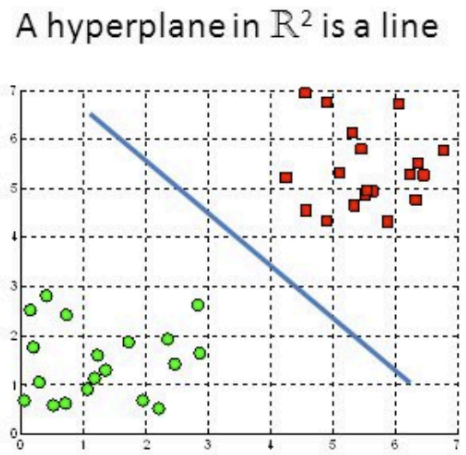


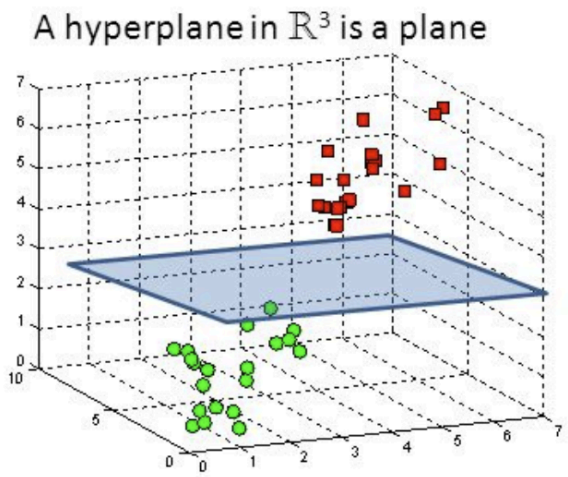

- **Tiled** partitioning is the norm for matrix and tensor ops: DRAM- and cache-efficient
- DRAM-to-disk: **pbdR**, **SystemDS**, **Dask arrays**; **SciDB**, **xarray** for n-d arrays; **CUDA** for DRAM-to-GPU

Dask: a $10{,}000^2$ array in $1{,}000^2$ chunks is 100 tasks of 8 MB each.

https://examples.dask.org/array.html

# Numerical optimization in ML




Many regression and classification models are a (constrained) **minimization** problem: logistic and linear regression, linear SVM, ...

**Empirical risk minimization:** the loss of the predictions over the labeled examples,

$$L(w) = \sum_{i=1}^{n} \ell(y_i, f(w, x_i))$$

**Generalized linear models** use an $f$ that is a scalar function of the distance to a hyperplane, $w^\top x$.

# Batch gradient descent

In many cases the loss $\ell$ is convex, so $L$ is too; a closed-form minimum is typically infeasible. **Gradient descent** is an iterative numerical procedure to find an optimal $w$:

1. Initialize $w$ to some $w^{(0)}$
2. Compute the **gradient** over all examples: $\nabla L(w) = \sum_{i=1}^{n} \nabla \ell(y_i, f(w, x_i))$
3. **Descend** along it (the update rule): $w^{(k+1)} = w^{(k)} - \eta \nabla L(w^{(k)})$
4. Repeat until $w$ is close to $w^*$: **convergence**

- The **learning rate** $\eta$ is a hyperparameter chosen by the user or by AutoML tuning
- The number of **iterations** (epochs) of BGD is a hyperparameter too

# The data access pattern of BGD at scale

The data-intensive computation in BGD is the **gradient**.

- In scalable ML, the dataset $D$ may not fit in DRAM; the model $w$ is typically **small and DRAM-resident**
- The gradient is a **SUM over vectors**, one per example
- At each epoch, **one filescan** over $D$ computes the gradient; the update of $w$ happens in DRAM
- The loss $L$ is also just a SUM, computed the same way; monitor it across epochs for convergence

**Q: What SQL operation is this reminiscent of?**

# The I/O cost of scalable BGD

The dataset as a row-store, one example (label and features) per tuple, six pages:

| Y | X1 | X2 | X3 |
|---|---|---|---|
| 0 | 1b | 1c | 1d |
| 1 | 2b | 2c | 2d |
| … | | | |

- A straightforward **filescan** access pattern for the SUM
- The same I/O behavior as the non-deduplicating project and the simple SQL aggregates

**I/O cost per epoch: 6 (read) + output pages (the final write of $w$).**

# Stochastic gradient descent

Two key cons of BGD:

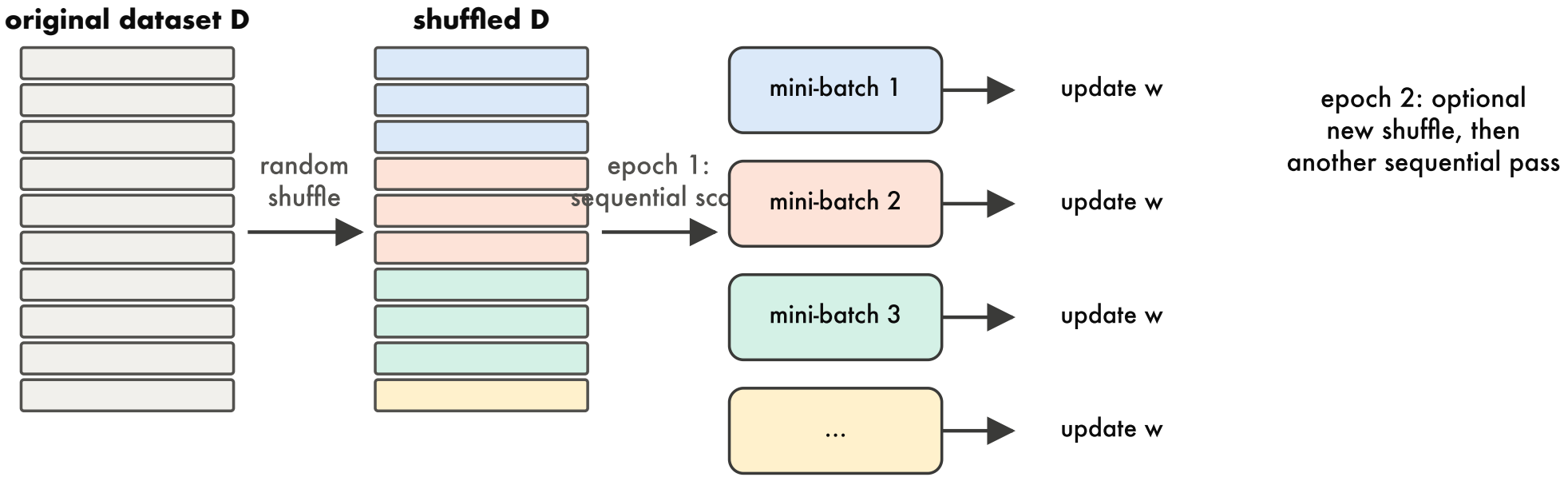
- Often **too many epochs** to reach the optimum
- Each update of $w$ needs a **full scan**: costly I/O

**SGD** mitigates both: use a **sample (mini-batch)** of $D$ to approximate the gradient instead of the full batch:

$$w^{(k+1)} = w^{(k)} - \eta \sum_{i \in B_k} \nabla \ell(y_i, f(w^{(k)}, x_i))$$

- Sampled **without replacement**: shuffle $D$ before each epoch, then one sequential pass over the mini-batches
- Also works for **non-convex** losses, especially deep learning; BGD does not: **SGD is the workhorse of modern ML.**

# The access pattern of scalable SGD




in SQL terms: ORDER BY RAND(), then one filescan per epoch

# The I/O cost of scalable SGD

- The **random shuffle** is non-trivial: an **external merge sort** (skipped in this course), typically 1 or 2 passes over the file
- Mini-batch gradients: **one filescan per epoch**; as the scan proceeds, count the examples seen and accumulate the per-example gradients for the current mini-batch
- Typical mini-batch sizes: tens to thousands, so **orders of magnitude more model updates** than BGD per epoch

| | |
|---|---|
| Total I/O cost per epoch | 1 shuffle cost + 1 filescan cost |
| In practice | shuffling **once** up front often suffices |
| Loss $L$ | computed as before, one filescan |

# Review questions

1. What are the four main regimes of scalable data access?
2. Explain one pro and one con of scaling with local disk versus scaling with remote reads.
3. A DataFrame is serialized as a 100 GB Parquet (columnar) file with 20 columns of the same fixed-length type. You compute a sum over 4 columns. What is the I/O cost in GB?
4. Which is the most flexible data layout for 2-D structured data?
5. You lay out a 1 TB matrix in tiled format with tiles of shape 2000 × 500. What is the I/O cost in GB of computing its full matrix sum?
6. Why does one epoch of SGD cost about the same I/O as one epoch of BGD, yet get far more done?

# Key takeaways

1. **Page in, compute, page out:** a file larger than DRAM is read through a cache of page frames with a replacement policy
2. **Count page I/Os**, not seconds; the hardware turns the count into latency
3. The **layout** must match the **access pattern**: row-store, column-store, and tiled layouts differ by orders of magnitude on the same query
4. **One filescan with small running state in DRAM** scales select, project, aggregates, matrix norms, and gradient descent
5. **GROUP BY** is the hard case: when the running state outgrows DRAM, split the data and conquer
6. **SGD** trades a shuffle for many more updates per pass, and is the workhorse of modern ML